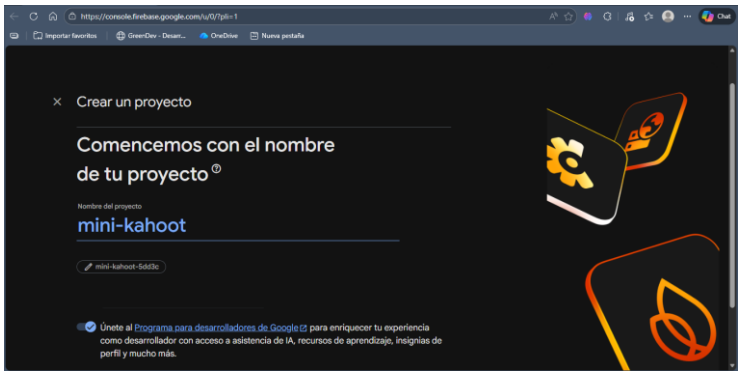
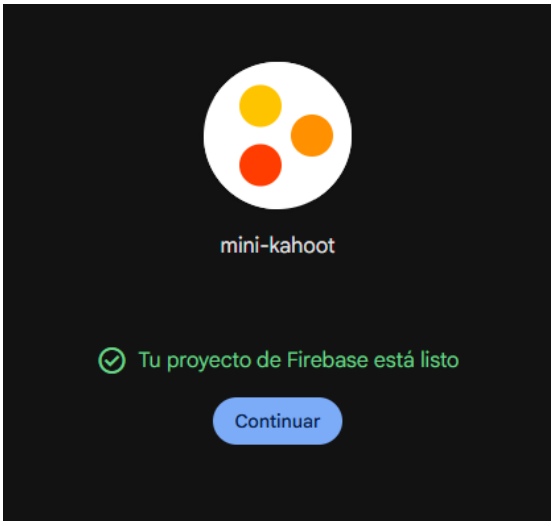
Firebase Setup and Flutter Connection

1: Creating the project in the Firebase Console



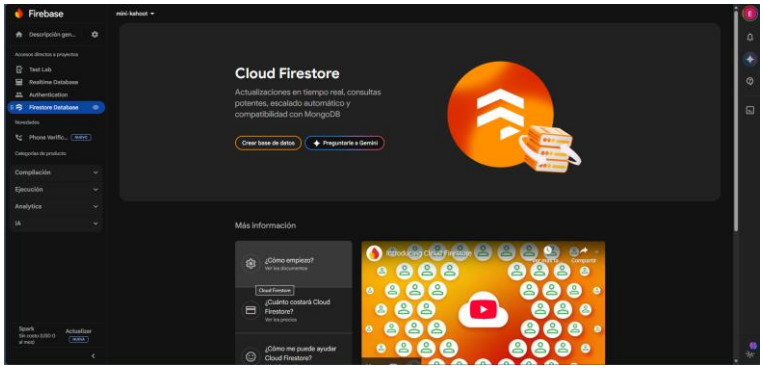
To begin, the Firebase Console was accessed to create a new project called "mini-kahoot". During the setup wizard, the architectural decision was made to enable Google Analytics. Although not strictly necessary for initial testing, it was activated to have an infrastructure ready for future project scalability, allowing us to monitor user interaction, game statistics, and connection metrics in real time.



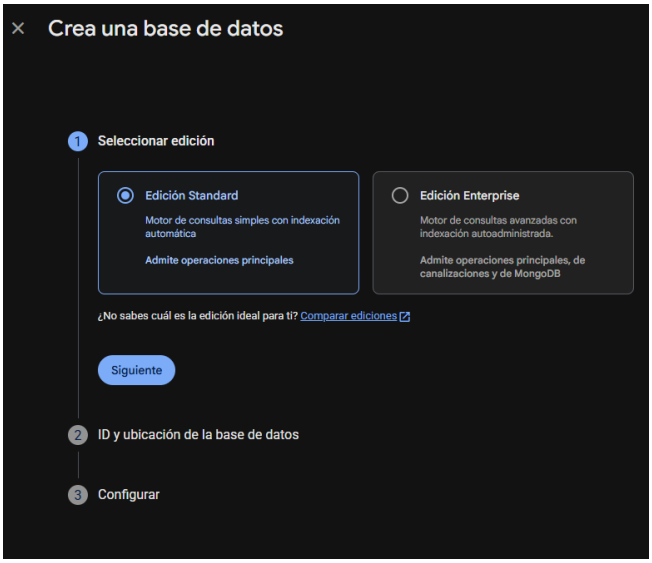
2. Database Configuration

For this project, the base document required the use of Firestore Database instead of Realtime Database, since the application logic is programmed using collections (games and players).

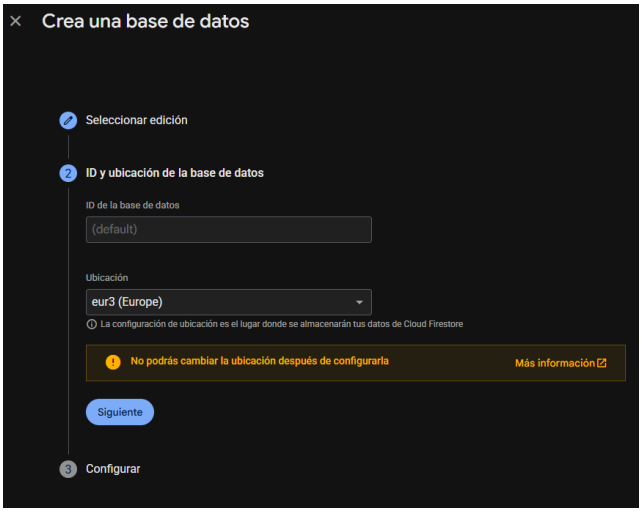
During creation in the Firebase console, it was configured with the following parameters:



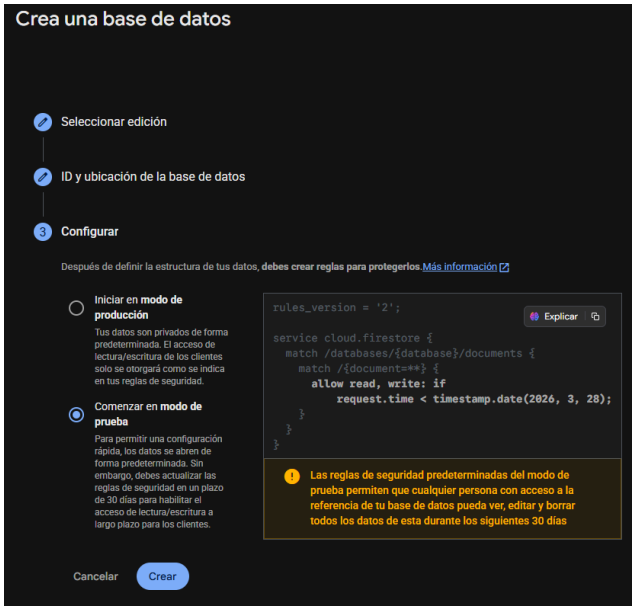
Edition: The Standard Edition was selected, which provides a simple query engine with automatic indexing, ideal and sufficient for the requirements of our mobile application.



Location: It was established in the European region (Europe-West) to optimize response times.

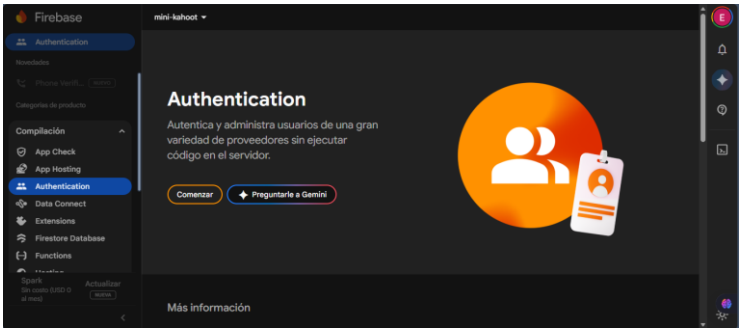


Security rules: The database was initialized in Test mode. This allows both the Server and Client apps to perform read and write operations without security locks during the development phase.

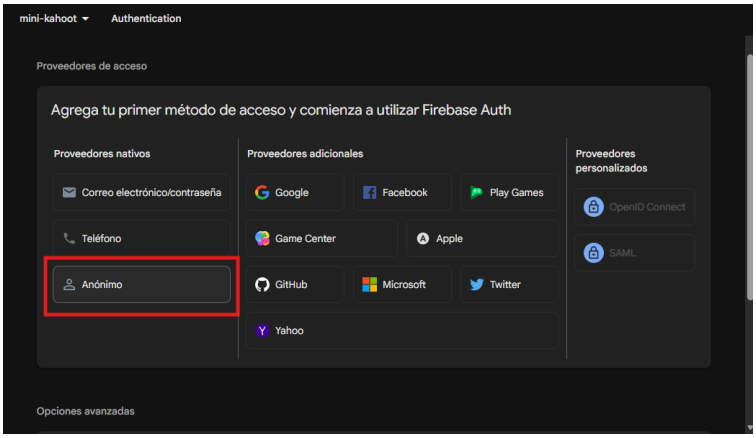


3. Enabling Authentication

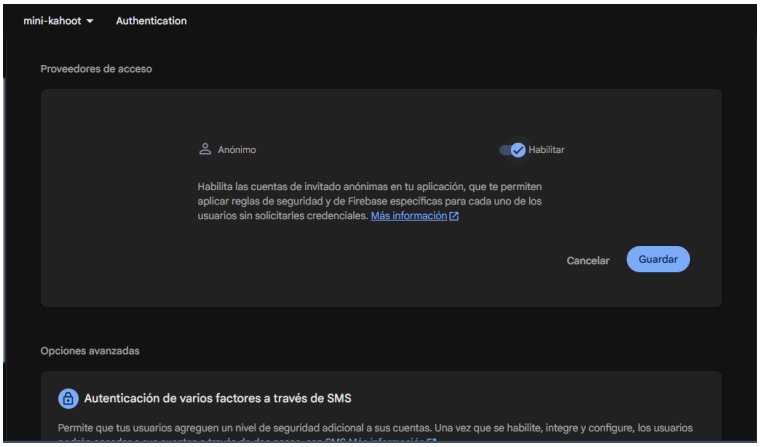
Go to the left menu in Firebase -> Authentication -> Get Started



To simulate the seamless experience of a real Kahoot, where students don't need to create an account or remember passwords to play, anonymous authentication has been implemented.



In the Authentication panel, under Sign-in methods, the Anonymous provider has been enabled. This allows Firebase to automatically assign a secure, temporary identifier to each teacher and student, enabling frictionless database access management for the end user.



4. Linking and Synchronizing the Flutter Project with Firebase

To connect the mobile application (server) to the newly configured Firebase project, the official FlutterFire tool was used. During the process, the installation was adapted to the needs of the Windows development environment (VS Code), following these steps:

1. Environment and CLI Preparation: First, the official Firebase command-line interface (Firebase CLI) was installed via Node Package Manager by running the command:

```
PS D:\FlutterData> dart pub global run flutterfire_cli:flutterfire configure
```

Next, the FlutterFire-specific tool was installed using Dart:

```
PS D:\FlutterData> dart pub global activate flutterfire_cli
```

2. Authentication: To allow the local environment to access the cloud projects, the firebase login command was executed, authorizing access through the Google account linked to the Firebase project.

```
PS D:\FlutterData> firebase login
Already logged in as erikespin12@gmail.com
```

3. Configuration and File Generation: Because Windows did not have the tool registered in its environment variables (PATH) by default, the configuration was executed by forcing the direct call through Dart with the following command:

```
+  Success! Logged in as erikespin12@gmail.com
PS D:\FlutterData\mini_kahoot_teacher> dart pub global run flutterfire_cli:flutterfire configure
i Found 2 Firebase projects.
? Select a Firebase project to configure your Flutter application with ›
❯ device-streaming-e78b620c (device-streaming-e78b620c)
  mini-kahoot-5dd3c (mini-kahoot)
  <create a new project>
```

During this interactive wizard, the terminal detected the available cloud projects and explicitly selected the "mini-kahoot-5dd3c" project. This process successfully completed, automatically generating the native configuration file firebase_options.dart. Thanks to this file, the application is now able to securely and platform-independently initialize Firebase to begin reading from and writing to the database.

```
PS D:\FlutterData\mini_kahoot_teacher> dart pub global run flutterfire_cli:flutterfire configure
i Found 2 Firebase projects.
✓ Select a Firebase project to configure your Flutter application with · mini-kahoot-5dd3c (mini-kahoot)
? Which platforms should your configuration support (use arrow keys & space to select)? ›
✓ android
✓ ios
✓ macos
✓ web
✓ windows
```

5. Demonstration of Connecting and Writing to Firebase (Server)

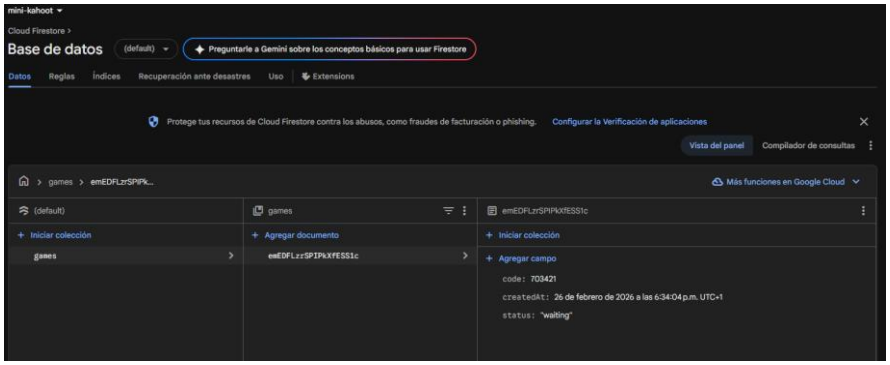
To verify that the bidirectional connection between the Flutter application and Firebase is working correctly, the Server application (Teacher's App) was run.

When you interact with the interface and press the "Create Kahoot" button, the application executes the createGame() method. This method generates a unique, 6-digit random numeric code and makes a write request to the Firestore database.

As a result, the successful creation of a new document within the root collection "games" is verified in real time. This document has been generated with a unique ID and contains the required data structure:

- code: The PIN to access the game (e.g., 703421).
- status: Initialized to "waiting", indicating that the server is waiting for connections.
- createdAt: The timestamp recorded by the Firebase server.

This test confirms that Firebase initialization is correct, that security rules allow writing, and that the data model has been implemented according to the project specifications.



6. Bidirectional and Real-Time Connection (Client - Server)

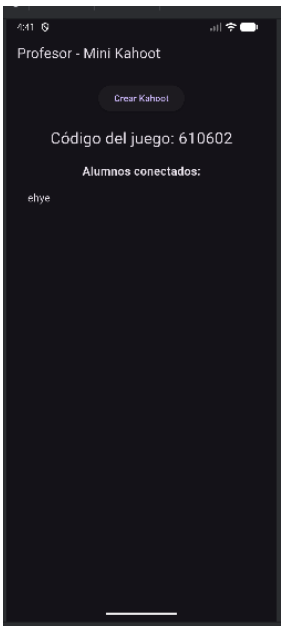
The Client application (Student App) has been developed and run on an Android emulator simultaneously with the Server app (Web).

By entering the generated PIN code and a username in the Client, the application writes to the players subcollection associated with that game in Firestore. Thanks to the use of a StreamBuilder in the Server application, it detects the database update in real time and automatically updates the interface, displaying the student's name in the list of connected players.

This test demonstrates the complete flow of bidirectional communication and real-time synchronization required in practice.

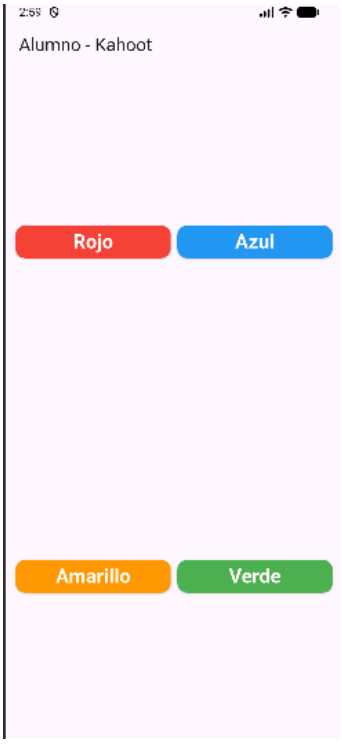
Client:

Server:



7. State Change and Screen Synchronization

The game control logic has been implemented. By pressing the "Start Game" button on the Server, the field of the game document in Firestore is updated from "waiting" to "playing". The Client application (Student) uses a that listens for changes in the main game document. Upon detecting that the status is "playing", the student's interface is automatically redrawn in real-time, hiding the waiting screen and displaying the four colored buttons (Red, Blue, Yellow, Green) ready for interaction.



8. Real-Time Interaction and Response Reception

The main game flow has been completed, fulfilling the final client-server interaction requirement. When the Student presses one of the colored buttons in the Client application, a write operation is executed in the subcollection of the current game document in Firestore. The Server application (Teacher) features a dedicated to listen to this subcollection in real-time. As a result, the teacher's interface instantly updates in the "Live Answers" section, displaying the player's name and the selected option (e.g., "Red") without needing to refresh the screen. This proves successful bidirectional communication and event synchronization.

Nombre:Erik Espinoza Kahoot-Flutter